Studio Nx

JS 123 — trimClipLeft()

2026-10-14

JS 123 — `trimClipLeft()` (왼쪽 끝 트리밍, 세 값 동시)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-arrange.js`
작업	S1-06 타임라인: 아홉 번째 함수 (FR-208 오디오 트리밍)
작성	Claude (AI), 2026-10-14, 사용자 검토
브랜치	`feature/phase1/s1-06-timeline`
다음	클립 복제(Ctrl+D)·분할 함수를 이어서 진행합니다

1. 하는 일

클립 **왼쪽 끝을 끌었을 때**의 새 값 세 개(`start_tick`·`length_tick`·`offset_tick`)를 한꺼번에 돌려줍니다. 오디오는 `offset_tick`(원본에서 건너뛰는 양)까지 같이 움직입니다.

- 줄이기: 오른쪽 끝과 같은 규칙. **최소 한 칸**, Alt면 1틱까지.
- 늘리기: **원본 처음**(`sourceStartTick`)과 **마디 0**에 닿을 때까지.

```
원본 오디오:  ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
클립:                ┌──────────────┐
                     ↑ offset       ↑ offset+length

왼쪽으로 늘리기  ◀──┤   → 원본 처음(offset 0)이나 마디 0 에서 멈춤
왼쪽으로 줄이기     ├─▶   → 한 칸(length = stepTicks)에서 멈춤
```

start	length	offset	원본 시작	끈 거리	한 칸	결과 start·length·offset
960	960	960	0	-960	960	0 · 1,920 · 0
960	960	960	0	-3,840	960	0 · 1,920 · 0 (마디 0에서 멈춤)
960	960	480	480	-960	960	480 · 1,440 · 0 (원본 처음에서 멈춤)
960	1,920	960	0	+960	960	1,920 · 960 · 1,920
960	960	960	0	+5,000	960	960 · 960 · 960 (한 칸에서 멈춤)

MIDI처럼 원본이 없으면 `sourceStartTick`에 `null`을 넣습니다. 그러면 늘리기 한계는 마디 0뿐이고 `offset`은 늘 0입니다.

2. 코드

```
export function trimClipLeft(startTick, lengthTick, deltaTicks, stepTicks, sourceStartTick) {
  const minLength = stepTicks;
  const minStart = sourceStartTick == null ? 0 : Math.max(0, sourceStartTick);
  const maxStart = startTick + lengthTick - minLength;
  let start = snapTick(startTick + deltaTicks, stepTicks);
  if (start < minStart) start = minStart;
  if (start > maxStart) start = maxStart;
  return {
    start: start,
```

```
    length: lengthTick - (start - startTick),
    offset: sourceStartTick == null ? 0 : start - sourceStartTick,
  };
}
```

최신 문법으로 쓰면 (공부용)

```
export const trimClipLeft = (startTick, lengthTick, deltaTicks, stepTicks, sourceStartTick) => {
  const minLength = stepTicks;
  const minStart = sourceStartTick == null ? 0 : Math.max(0, sourceStartTick);
  const maxStart = startTick + lengthTick - minLength;
  const start = Math.min(Math.max(snapTick(startTick + deltaTicks, stepTicks), minStart), maxStart);
  return { start, length: lengthTick - (start - startTick), offset: sourceStartTick == null ? 0 : start
- sourceStartTick };
};
```

최신 문법	실제 코드	뜻
화살표 함수	`function` 선언	같은 동작입니다
중첩 삼항·`Math.min/max`를 한 줄에	`if` 두 줄	읽기 쉽게 풀어 쓴 것뿐입니다
`??` (null 병합)	`== null ?`	`null`과 `undefined`를 함께 거르되 `0`은 살립니다

3. 왜 이렇게

선택	이유
`offset`을 넣지 않고 결과로 돌려줌	오프셋은 `start - 원본시작`으로 저절로 정해집니다. 넣을 필요가 없습니다
`sourceStartTick` 하나로 오디오·MIDI 구분	원본이 없으면 `null`. 그러면 마디 0 한계만 남고 오프셋은 0이라 MIDI 동작이 됩니다
늘리기 = `minStart`로 clamp	원본 처음(`max(0, sourceStart)`)보다 왼쪽은 원본에 없는 부분이라 막습니다
줄이기 = `maxStart`로 clamp	`length = stepTicks`가 되는 지점입니다. 오른쪽 끝 함수와 최소 규칙이 같습니다
격자 맞춘 뒤 clamp	결과가 격자 근처에서 한계에 붙습니다. 한계값이 격자 밖이어도 정확히 한계에서 멈춥니다
세 값을 항상 함께 돌려줌	`start`가 움직인 만큼 `length`가 반대로, `offset`이 같이 움직여야 오디오가 제자리에 고정됩니다

4. 시험 (Node에서 직접 실행, 13개 통과)

확인	결과
제자리 · 늘리기(960→0, 길이 1,920, offset 0)	✓
원본 처음 한계(sourceStart 480에서 멈춤) · 마디 0 한계	✓
왼쪽 줄이기(1,920−960→start 1,920·길이 960·offset 1,920)	✓
한 칸 밑으로 줄여도 멈춤 · Alt면 1틱까지	✓
MIDI(`sourceStartTick = null`): 마디 0까지 자유·offset 0	✓
step 0·delta `NaN` → 오류	✓